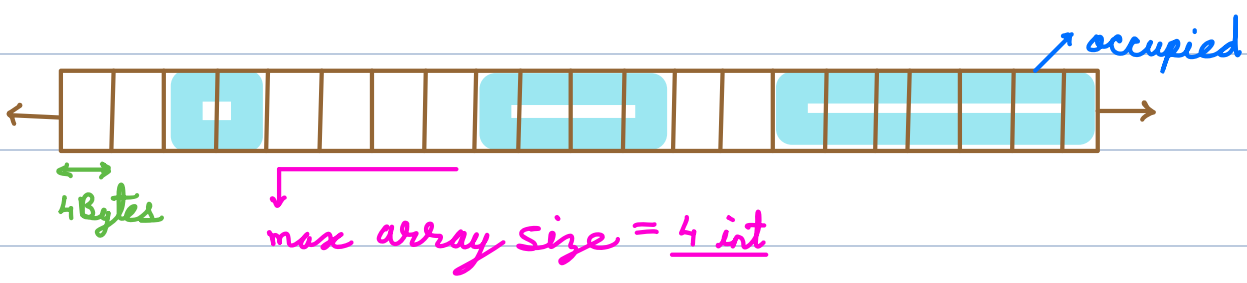
Agenda:

1. Introduction + Use of Linked List over array
2. Access K'th index node
3. Search for element X
4. Insert element X at position K
5. Delete first occurrence of X

Issues with Array

Fixed size → use array list for dynamic size.



LL → Not continuous memory allocation ⇒ large size can be created.

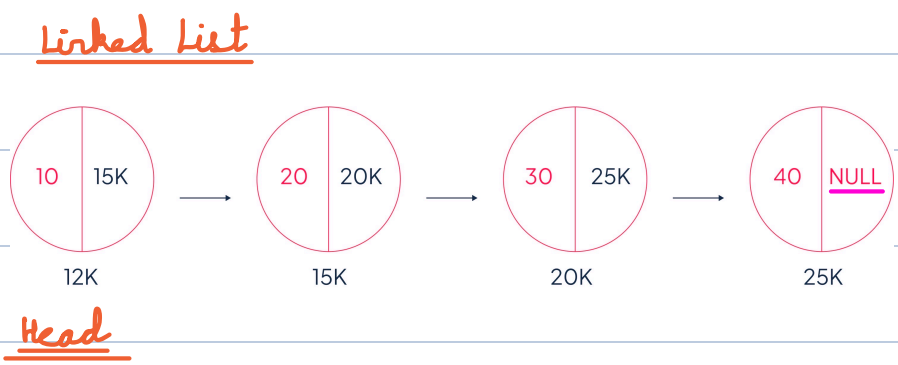
Linked List

- A linear data structure that can utilise all the free memory
- We need not have continuous space to store nodes of a Linked List

Node:

- It has a data section where the data is present
- A next pointer which points to next element of the linked list

```
class Node{
    int data;
    Node next;
    Node(int x){
        data = x;
        next = null;
    }
}
```

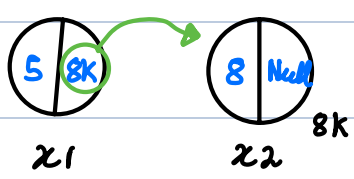


Node x1 = new Node (5);

Node x2 = new Node (8);

Representation:

- The first node of the linked list is called head
- Any linked list is represented by its first node

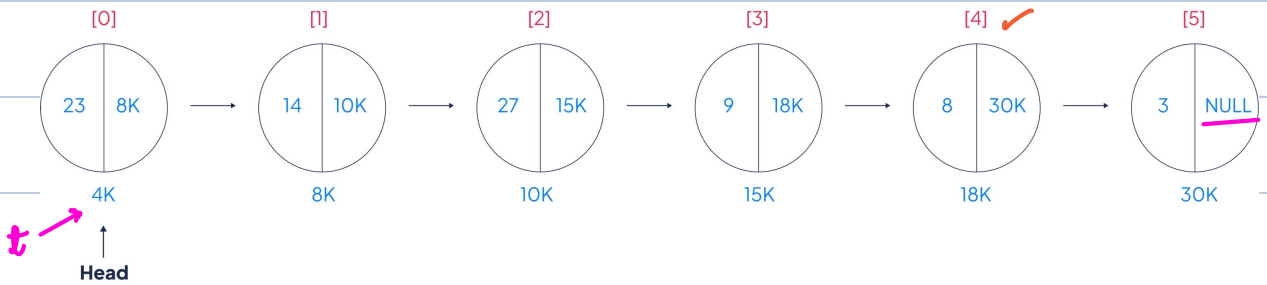


x1.next = x2

Access in Linked List

A[i] → TC = O(1)

Q1: Given a linked list, find its K'th index node [**0 <= K < N**].



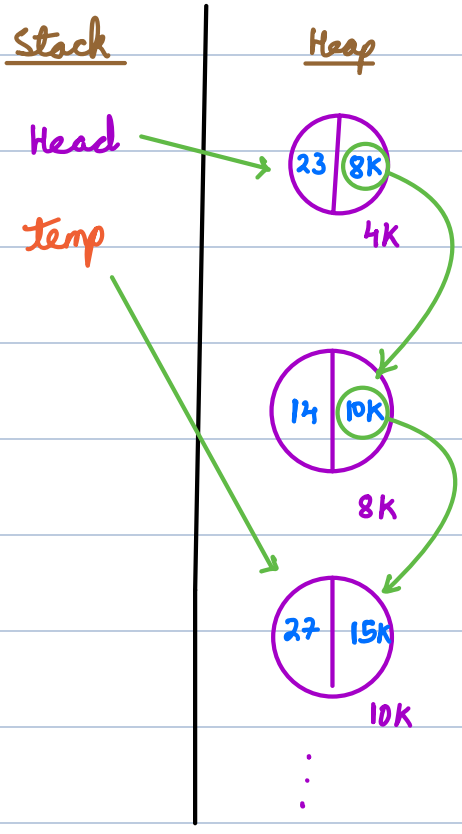
Example: K = 4 → 8

K = 0 → 23

```
Node temp = Head          // shallow copy
for i → 1 to K {
    temp = temp.next
}
return temp.data
```

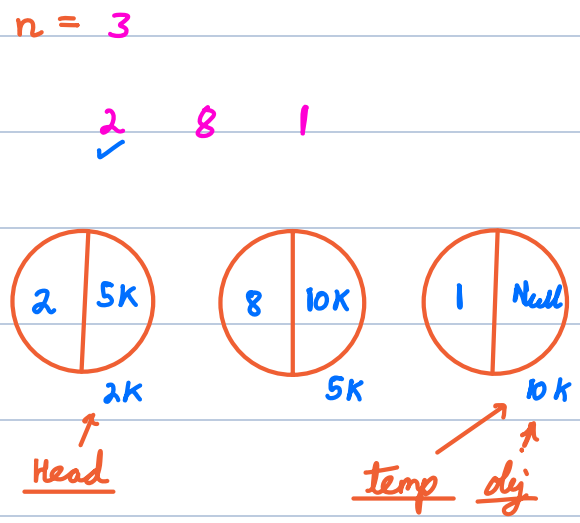
TC = O(K)

for K^{th} element



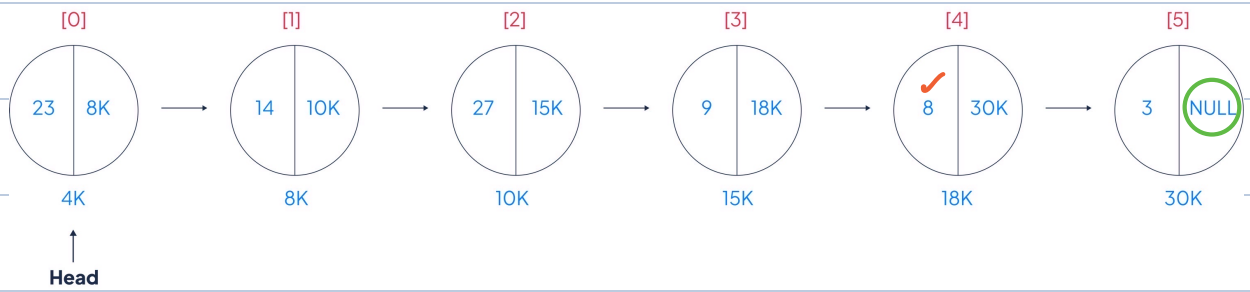
creation of Linked List

```
// --- Create a Linked List ---
int n = sc.nextInt();    // n > 0
Node Head = new Node( sc.nextInt() );
Node temp = Head;    // Shallow Copy
for(int i = 1; i < n; i++) { i → {1,2}
    Node obj = new Node( sc.nextInt() );
    temp.next = obj;
    temp = temp.next;
}
```



Search in Linked List

Q2: Given a linked list, check if an element X is present in the linked list.



Example: search(8) → true

search(5) → false

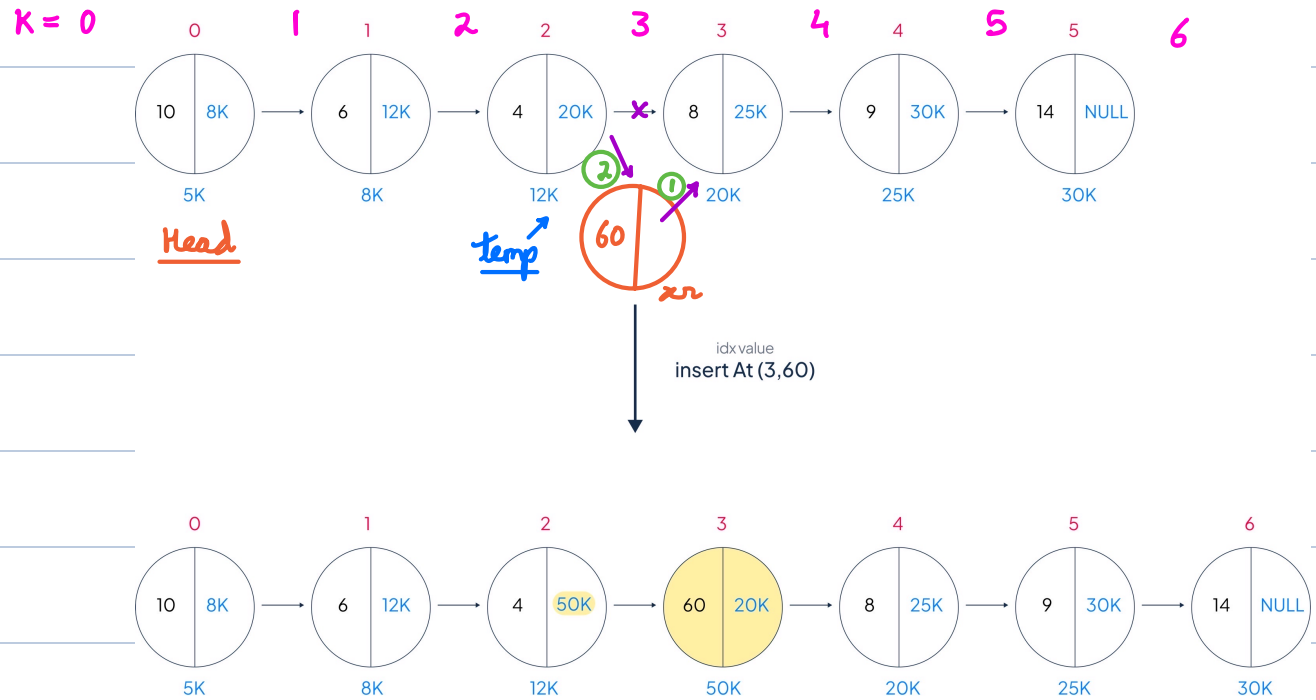
```
Node temp = Head                    // Empty list → Head = Null
while (temp != Null) {
    if (temp.data == K)   return true
    temp = temp.next
}
return false
```

TC = O(N)

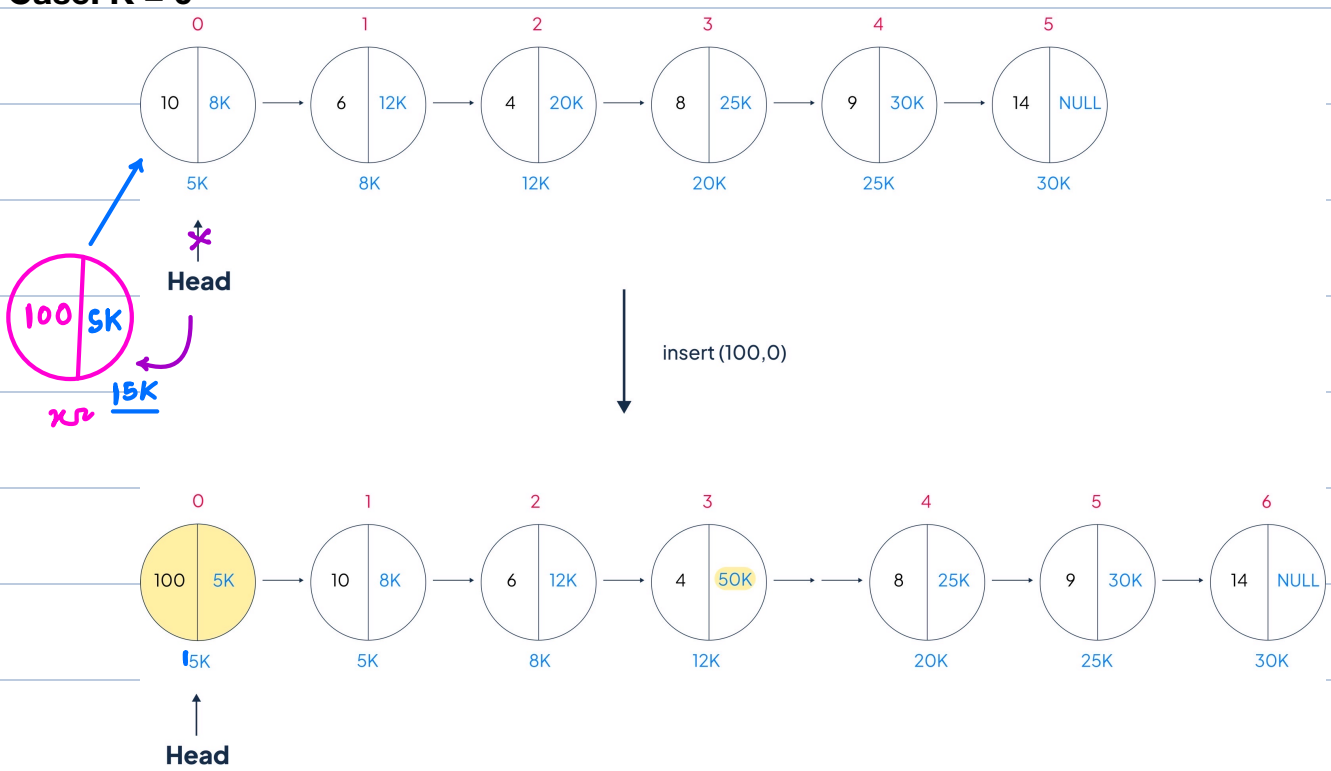
Insertion in Linked List

Q3: Insert an element X at index K in the given linked list.

$0 <= K <= N$



Case: K = 0



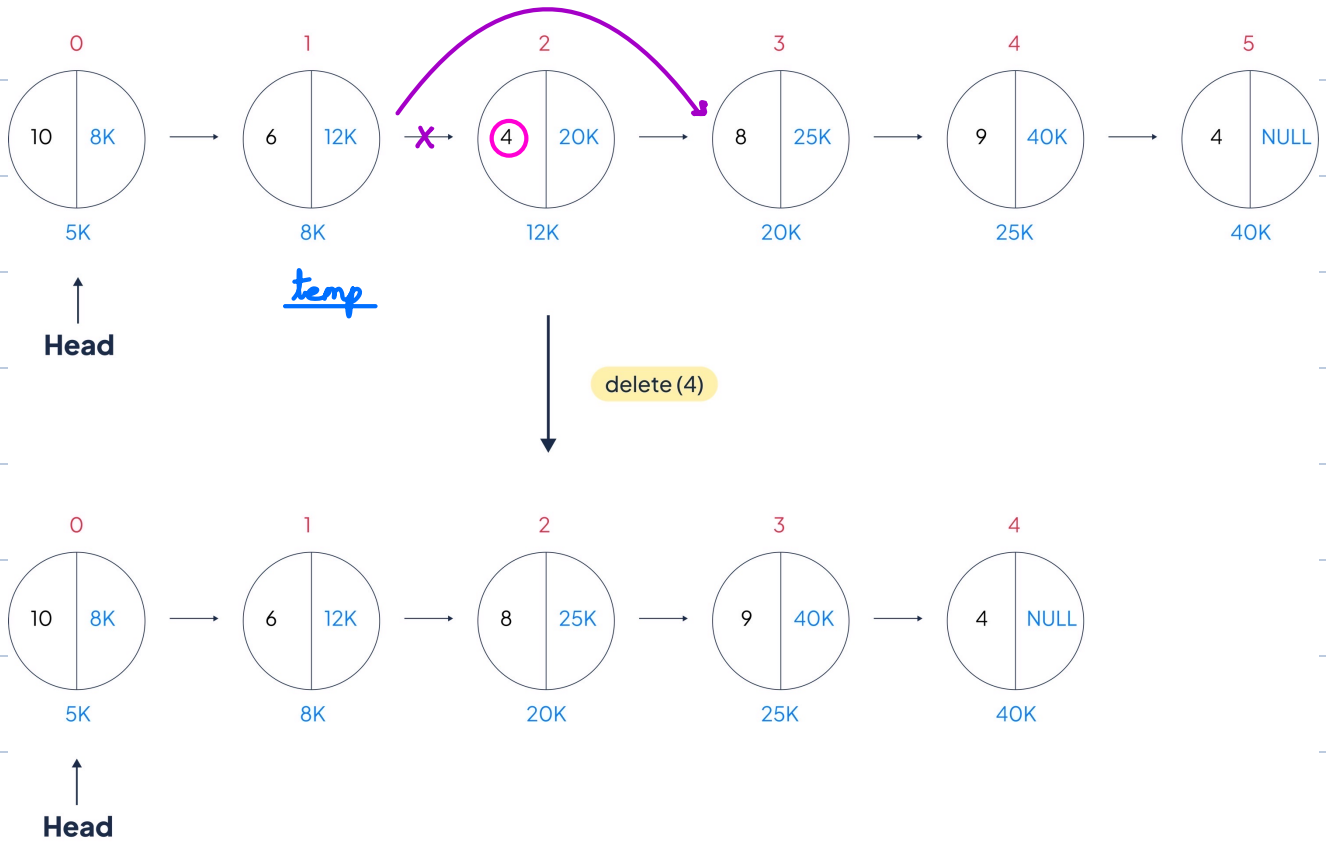
```
Node xr = new Node (X)
if (K == 0) {
        xr.next = Head
        Head = xr
        return Head
}
temp = Head
for i → 1 to (K-1) {
        temp = temp.next
}
xr.next = temp.next
temp.next = xr
return Head
```

TC = $\underline{O(K)}$

Deletion in Linked List

Q4: Delete the first occurrence of an element X in the given linked list.

If the element is not present then no change.




```
if (Head.data == x) {
        Head = Head.next
        return Head
}

temp = Head
while (temp.next != null) {
        if (temp.next.data == X) {
                temp.next = temp.next.next
```

```
        break
    }
    temp = temp.next
}
return Head
```

TC = O(N)

H.W → Delete all occurrences of x.

What will be the output of following code:

```java
public static boolean search(Node head, int x) {

    while(head.next != null) {
        if(head.data == x)
            return true;
        head = head.next;
    }

    return false;
}

public static void main(String[] args) {

    Node head = new Node(10);
    head.next = new Node(20);

    System.out.println(search(head, 20));
}
```

20

10 → 20 → null

Head

10 → 20

Head

→ false

What will be the output of following code:

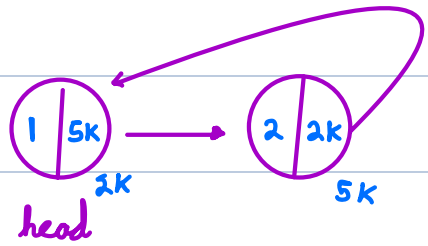
```
public static void main(String[] args) {

    Node head = new Node(1);
    head.next = new Node(2);

    head.next.next = head;

    Node temp = head;

    while(temp != null) {
        System.out.print(temp.data + " ");
        temp = temp.next;
    }
}
```



1 2 1 2 . . .